

Caso de Uso: Exact Number — AWS

Raúl Jiménez de la Cruz¹

Resumen— Este artículo expone una arquitectura desacoplada que implemente un caso de uso de interés mediante servicios AWS

Palabras clave— AWS, Despliegue, Caso de Uso.

I. INTRODUCCIÓN Y OBJETIVOS

Una de las razones por las que se migran microservicios hacia AWS es para ganar escalabilidad y eficiencia operativa, ya que, en caso de desplegar a AWS Lambda, las empresas pagan solo por el tiempo de ejecución real, eliminando el costo de mantener servidores ociosos durante picos de tráfico bajos y asegurando elasticidad horizontal[1].

Entonces, se propone hacer un ejercicio demostrativo, en el que se toma una aplicación monolítica existente, con una determinada funcionalidad, para construir alrededor una arquitectura *cloud*.

Dicha aplicación monolítica se trata de un *simulador de cálculo* de cifra exacta, similar al juego del programa de televisión “Cifras y Letras”. A pesar de que el objetivo de la presente memoria no es detallar su funcionamiento, sí resulta crucial que el lector considere que es una aplicación montada en Java, usando el marco Spring Boot, aunque se ejecute por consola¹. Nótese que el desarrollo del microservicio se realizó empleando Maven como gestor de dependencias, a través del fichero `pom.xml` lo que nos permitirá que la integración futura con Lambda sea más sencilla, sin romper código existente.

II. REQUISITOS Y ARQUITECTURA

A. Sondeo y Selección de Servicios AWS

Como se comentó anteriormente, se parte de una aplicación Java que funciona, actualmente, por terminal. Entonces, el primer servicio o componente a proponer, naturalmente, es migrar la lógica de negocio a un **Lambda**. De esta manera, la JVM pasará de ser controlada por nosotros, en entorno local, a ser controlada por AWS. Dicho de otra manera, no hay necesidad de administrar la infraestructura, y, además, permite gestionar las ejecuciones del microservicio (usar únicamente el tiempo de ejecución necesario). Además, puede permitir escalar el servicio automáticamente.

De todas formas, si fuese para nosotros una preocupación la saturación del servicio (la Lambda propiamente dicho), se monta antes de la lógica de negocio una cola de mensajes (**SQS**), que unificaría la vista (*frontend*) con Lambda. Su rol sería actuar como un búfer, para permitir reintentos de cálculos, en

caso de que saliesen mal, o controlar muchos cálculos simultáneos.

En cualquier caso, si queremos comunicar los resultados de estas simulaciones por el resto de la arquitectura, imprimir los resultados por terminal deja de ser una opción. Para ello, se propone emplear **DynamoDB** como capa de persistencia donde almacenar, de forma estructurada, los resultados de cada ejecución del minijuego. Algunos campos a persistir serán los números de entrada, el número objetivo, un listado de operaciones empleadas, entre otros.

Con *inputs* sucede algo similar, el usuario, naturalmente, no tendrá acceso a la terminal donde se ejecuta la Lambda para especificar los parámetros del cálculo. Por ello, lo primero de todo es establecer una comunicación entre el usuario y el servicio. Por consecuente, usar **API Gateway** habilita la definición de un *endpoint*, para tener una URL limpia y “hablar” con la Lambda. Luego, podemos usar **Cognito** para gestionar la autenticación de usuarios, y, así, cada usuario puede ver los resultados de sus simulaciones. Si fuese necesario establecer un *rate limit* para no abusar de la API, un Firewall (**WAF**) puede ayudar a protegerla.

Ahora, si queremos que el usuario pueda elegir los números de forma cómoda, puede complementar con una web estática, alojada en un **S3**, donde se especifiquen los parámetros (p.e usando desplegables). Luego, de cara a realizar mejoras en una siguiente iteración –por ejemplo, si la web crece en complejidad– se puede incluir un **CloudFront** (CDN) para acelerar la entrega de contenido. Dicho de otra manera: *el frontend estático vive en S3, CloudFront lo sirve como CDN*.

Además, se proponen dos *features* nuevas:

- Enviar los resultados de forma personalizada vía e-mail, utilizando, para ello, servicio de correo (**SES**). Si en un futuro se quisiese enviar los resultados de otra manera, se puede acoplar inmediatamente antes un sistema de notificaciones simple (**SNS**), que puede actuar como un *middleman*: ayuda a separar responsabilidades o elegir otro medio de envío de notificaciones, como un SMS.
- Resúmenes semanales con **EventBridge Scheduler**. Por ejemplo, permite ejecutar el microservicio, por ejemplo, cada semana, para tareas de análisis de datos o reportes (un email semanal tipo *digest* con las partidas de esa semana). Eliminaría la necesidad de tener un servidor que aloje esa Lambda encendido todo el rato, ya que sólo se dispararía cuando el Scheduler lo requiera.

En la Figura 2 se presenta un diagrama de arquitectura detallado, con la iconografía de AWS, aunque

¹Estudiante del MUBDCN (UCLM), e-mail: Raul.Jimenez8@alu.uclm.es

¹En el documento de propuesta de caso de uso se explica con más detalle el microservicio

el flujo y descripción se detallará en la siguiente sección.

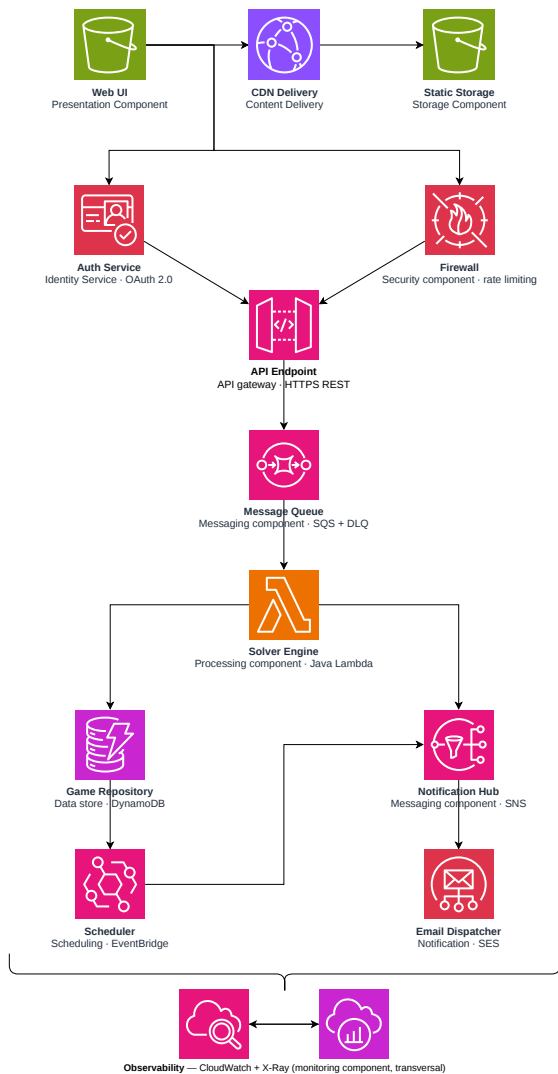


Fig. 1: Diagrama de Arquitectura

B. Descripción de la Arquitectura y Flujo

La arquitectura se basa en un flujo desacoplado de microservicios sobre AWS. La entrada principal se realiza a través de API Gateway, que actúa como capa de exposición del sistema. A continuación, las peticiones son validadas mediante Cognito (autenticación), filtradas por WAF (seguridad perimetral) y enrutadas hacia servicios backend.

El flujo típico de una petición sincrónica es el siguiente:

- Cliente realiza una petición HTTP a API Gateway.
- API Gateway valida y reenvía la petición.
- Cognito autentica el token de usuario.
- WAF inspecciona tráfico entrante.
- Lambda procesa la lógica de negocio.
- SQS desacopla procesos asíncronos.
- DynamoDB almacena y recupera datos persistentes.

- CloudWatch y X-Ray registran trazas y métricas.

Este diseño prioriza escalabilidad horizontal, tolerancia a fallos y observabilidad completa del sistema.

III. IMPLEMENTACIÓN

A. Preparación del microservicio

El primer paso natural fue adaptar el microservicio actual para que fuese compatible con lambda de AWS. Para ello fue necesaria una refactorización total del monolito existente. Lo primero que se hizo fue añadir las integraciones necesarias para que pueda funcionar en AWS. Para ello, se comenzó por la creación de repositorios java, uno para mostrar los resultados en memoria, ideal para local y otro para guardar los resultados en una base de datos DynamoDB. Luego, se codificó un handler para Lambda que dispare automáticamente la ejecución del servicio.

Sin embargo, faltaba el paso más importante, que es desacoplar Spring Boot del monolito, al menos para la parte de Lambda. Las dependencias más básicas de Spring Boot, como `spring-boot-starter-web` y `spring-boot-starter-thymeleaf`, arrastran un servidor Tomcat embebido, el motor de plantillas Thymeleaf, el contexto de Spring y el ciclo de arranque de Spring Boot. Uno de los problemas de este enfoque es que cuando Lambda quiera instanciar `SolverLambdaHandler ::handleRequest` no pasará por `main()`, y Spring Boot nunca arrancará.

El otro problema es el binario JAR resultante: todo lo listado anteriormente resultarán en unos ¿30MB de dependencias que Lambda no necesita realmente, por lo que puede afectar al arranque de la ejecución, por no mencionar la estructura generada, la cual Lambda no entiende (Maven genera `BOOT-INF/classes` y `BOOT-INF/lib`, y Lambda espera una estructura más estándar, como `uber-jar` plano).

La solución que se decidió implementar fue desacoplar Spring Boot como motor de arranque para Lambda. Para ello, el monolito se convirtió en una aplicación multi-módulo, cada uno con sus propias dependencias, de tal manera que uno para el CLI (que mantiene Spring Boot) y uno para Lambda (que es un JAR plano con solo las dependencias necesarias).

Una vez hechos los cambios oportunos, generamos el JAR, compilando desde proyecto raíz con `mvn clean install` y, luego, `mvn clean package -pl lambda -am` para generar el JAR que corresponde únicamente con el módulo de Lambda (que es el que se subirá a AWS).

De esta manera, ya tenemos el monolito preparado para migrar hacia AWS².

²Dicho monolito será compartido en el repositorio del proyecto, mediante una copia del código fuente. No obstante, el proyecto original se encuentra en `monolith-to-aws`

B. Verificación de Lambda y DynamoDB

Ahora, antes de preparar el despliegue automático con Terraform, verificamos si, desde nuestra máquina, tenemos acceso a los recursos *cloud*. Para ello, configuramos las credenciales, en el archivo `~aws/credentials`³, para poner las credenciales del Learner Lab. Con la herramienta `awscli` comprobamos que tenemos acceso a todo:

```
$ aws dynamodb list-tables --region us-east-1
{
  "TableNames": [
    "CalculationResults"
  ]
}
$ aws lambda list-functions --region us-east-1
{
  "Functions": [
    {
      ...
    },
    {
      "FunctionName": "cifras-lambda",
      "Runtime": "java21",
      "Role": "arn:aws:iam:.../LabRole",
      "Handler": "com.kangoo.cyl.lambda.SolverLambdaHandler::handleRequest",
      ...
      "Environment": {
        "Variables": {
          "TABLE_NAME": "CalculationResults"
        }
      },
      ...
    }
  ]
}
```

Una vez verificamos los recursos, levantamos la infraestructura Terraform. Después, podemos probar que el microservicio funciona correctamente, usando el apartado “Test”, con este payload:

```
{
  "numbers": [100,75,25,2,3,1],
  "target": 301
}
```

Y así comprobamos que DynamoDB guarda los datos en la base de datos, con todos los parámetros.

C. API Gateway

Por consecuente, ya es el paso de incluir API Gateway, para definir una comunicación propiamente dicha con el microservicio, y no a través de una interfaz de prueba en la consola de AWS.

Entonces, el primer paso es refactorizar el Handler, para que, en vez de un DTO cerrado que podemos pasarle a través de “Test”, le pasemos un objeto `<Map<String, Object>`, y que dicho *handler* se centre en procesar la entrada.

El resto del procesado se queda igual. El resto se tratará en codificar en los ficheros de terraform, a grandes rasgos, la definición del servicio de API Gateway, el endpoint a usar y la integración con Lambda. Entonces, una vez se puede aplicar los cambios, probamos a hacer una petición HTTP, tomando como base la URL que nos ha dado el *output* del despliegue:

```
curl -X POST \
https://XXXXXXX.execute-api.us-east-1.amazonaws.com/solve \
-H "Content-Type: application/json" \
-d '{"numbers": [2,10,4,6,3,2], "target": 267}' | jq
{
  "target": 267,
  "distance": 0,
  "operationsNeeded": 5,
  "operations": [
    "2 + 10 = 12",
    "4 * 6 = 24",
    "24 - 2 = 22",
    "12 * 22 = 264",
    "3 + 264 = 267"
  ]
}
```

D. Graphviz Resultante

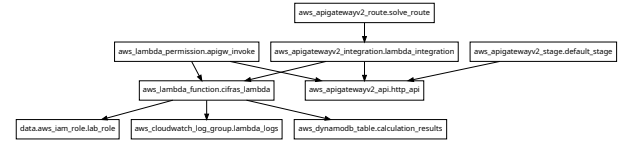


Fig. 2: Diagrama Graphviz de Terraform

IV. ANÁLISIS DE COSTES Y PRESTACIONES

A. Tiempos de Respuesta por Servicio

Los valores indicados corresponden a latencias de percentil P50 y P99 típicas en la región **us-east-1**, bajo carga moderada (< 500 req/s).

Servicio	P50 (ms)	P99 (ms)	Observaciones
S3 (GET objeto ¡1 MB)	15	80	Varía con tamaño del objeto
Cognito (token)	50	200	Incluye validación JWT
WAF	1	5	Inspección inline
API Gateway (REST)	10	50	Excluye back-end
SQS (SendMessage)	5	20	Standard queue
Lambda cold start (Java)	800	3000	Primera invocación / sin Snap-Start
Lambda warm (Java)	20	150	Ejecuciones subsiguientes
DynamoDB (read/write)	2	10	Single-digit ms garantizado
SNS (publish)	5	30	Entrega asíncrona
SES (SendEmail)	100	500	Entrega externa
EventBridge (rule)	500	2000	Invocación target
CloudWatch (PutMetricData)	20	100	Asíncrono

Tabla I: Latencias típicas por servicio (us-east-1)

B. Latencia Extremo a Extremo (E2E)

El tiempo de respuesta E2E de una petición sincrónica típica es:

$$T_{E2E} = T_{S3} + T_{Cognito} + T_{WAF} + T_{APIGW} + T_{SQS} + T_{Lambda} + T_{DDB}$$

$$T_{E2E}^{P50} \approx 15 + 50 + 1 + 10 + 5 + 20 + 2 = 103 \text{ ms}$$

$$T_{E2E}^{P99} \approx 80 + 200 + 5 + 50 + 20 + 150 + 10 = 515 \text{ ms}$$

³ Asumimos sistema de archivos de un SO basado en UNIX

Nota: El cold start de Lambda Java puede elevar el P99 hasta aproximadamente 3,5 segundos en el primer arranque. Se recomienda activar Lambda SnapStart para reducirlo por debajo de 100 ms.

C. Throughput y Escalabilidad

Servicio	Límite por defecto	Ampliable
API Gateway	10 000 req/s	Sí, vía AWS Support
Lambda concurrencia	1 000	Sí, hasta cientos de miles
SQS	Ilimitado	—
DynamoDB	Ilimitado (on-demand)	—
SES	14 emails/s	Sí, tras sandbox
Cognito	1 200 req/s	Sí

Tabla II: Límites de escalabilidad por servicio

D. Disponibilidad y SLA

Servicio	SLA	Downtime máx./año
S3	99.9 %	8,7 h
API Gateway	99.95 %	4,4 h
Lambda	99.95 %	4,4 h
DynamoDB	99.999 %	5,3 min
SQS	99.9 %	8,7 h
Cognito	99.9 %	8,7 h
SES	99.9 %	8,7 h
SNS	99.9 %	8,7 h

Tabla III: SLAs de disponibilidad declarados por AWS

$$A_{sistema} \approx 99,7\%$$

E. Puntos de Atención de Rendimiento

- Cold start Java: principal cuello de botella. Mitigación: SnapStart o Provisioned Concurrency.
- DLQ: configurar alarmas CloudWatch sobre métricas de mensajes.
- Throttling de Cognito: puede requerir caching de tokens.
- SES Sandbox: limitado a 14 emails/s hasta verificación.
- Consistencia eventual DynamoDB: lecturas consistentes incrementan coste.

V. ESTIMACIÓN ECONÓMICA — 12 MESES (US-EAST-1)

A. Hipótesis de Carga

Las estimaciones se basan en los siguientes supuestos de uso moderado-producción:

Parámetro	Valor/mes	Justificación
Peticiones API	10M	~333K/día
Invocaciones Lambda	10M	1:1 con API
Duración media Lambda	500 ms	Solver Java
Memoria Lambda	512 MB	—
Mensajes SQS	20M	Request + respuesta
Lecturas (RCU) DynamoDB	50M	5 reads/petición
Escrituras (WCU) DynamoDB	10M	1 write/petición
Almacenamiento DynamoDB	10 GB	—
Datos en S3	50 GB	UI + assets
Solicitudes S3 PUT/GET	5M	—
Usuarios Cognito	10K	MAU
Emails SES	200K	2 % de peticiones
SNS	500K	—
EventBridge reglas	10	Programadas
CloudWatch métricas	50	—
Logs CloudWatch	20 GB	—
Trazas X-Ray	10M	1 por petición
WAF	1 ACL, 3 reglas	Básico

Tabla IV: Hipótesis de dimensionamiento

VI. ENLACE AL REPOSITORIO

Todos los artefactos involucrados en el proyecto *cloud* descrito a lo largo de la presente memoria se encuentran en el siguiente repositorio:

`cloud-systems-services-aws-exact-number`⁴

REFERENCIAS

- [1] AWS Editorial Team, “Migrando una aplicación monolítica para la arquitectura serverless y microservicios — blog de amazon web services (aws),” Nov 2019.

⁴<https://github.com/RaulJD1CRUZ/cloud-systems-services-aws-exact-number>